

Customer Feedback Intelligence

Study Notes — ML Engineering, FastAPI, Testing, Docker & CI/CD

Big Idea

This project was not just about training a sentiment model.

We learned how to take a machine-learning model and turn it into a **real usable software service**:

Data → **ML Model** → **API** → **Tests** → **Docker** → **CI/CD** → **Deployment**

1. What Did We Build?

A customer feedback analysis API.

Input

```
{
  "feedback": "I was charged twice and support has not replied."
}
```

Output

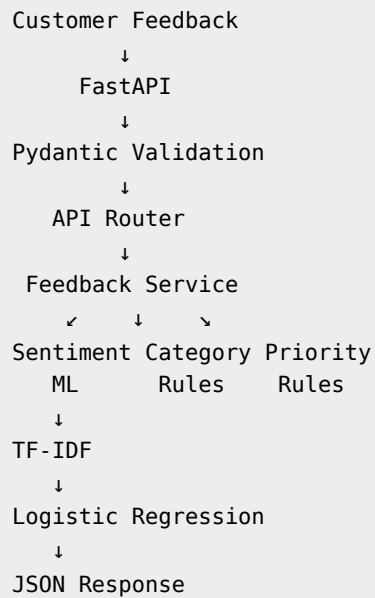
```
{
  "sentiment": "negative",
  "category": "billing",
  "priority": "high"
}
```

Current Intelligence

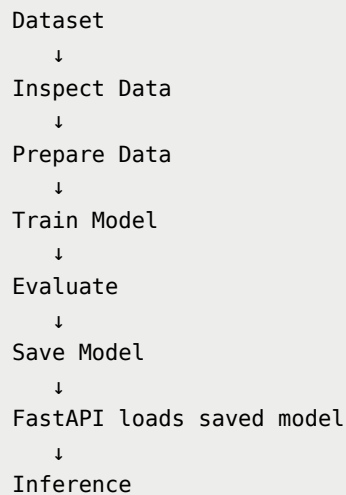
Component	Approach
Sentiment	Machine Learning

Component	Approach
Category	Business rules
Priority	Business rules

2. Whole Project at a Glance



Training happens separately



Exam point

Training = learning from labelled data.

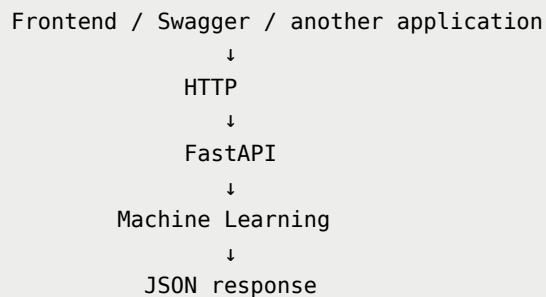
Inference = using an already-trained model to predict new data.

3. API Basics

What is an API?

An **Application Programming Interface** allows software systems to communicate.

In this project:



HTTP Methods We Used

Method	Purpose	Project Example
GET	Retrieve information	/health
POST	Send data for processing	/feedback/analyse

GET Example

```
GET /health
```

Response:

```
{  
  "status": "healthy"  
}
```

POST Example

POST /feedback/analyse

Body:

```
{  
  "feedback": "The application keeps crashing."  
}
```

4. FastAPI

What is FastAPI?

FastAPI is a Python web framework for building APIs.

```
from fastapi import FastAPI  
  
app = FastAPI()
```

`app` is our API application.

Why did we choose FastAPI?

- Python-based
- good fit for ML applications
- fast to develop
- automatic validation

- integrates with Pydantic
- automatically creates Swagger documentation
- supports REST APIs

Running FastAPI

```
uvicorn app.main:app --reload
```

Break it down

Part	Meaning
<code>uvicorn</code>	Starts the web server
<code>app.main</code>	<code>app/main.py</code>
<code>:app</code>	FastAPI variable called <code>app</code>
<code>--reload</code>	Restart server when code changes



5. Uvicorn

What is Uvicorn?

FastAPI defines the API, but something still has to **listen for HTTP requests**.

That is Uvicorn's job.

```
Browser
  ↓
HTTP Request
  ↓
Uvicorn
  ↓
FastAPI
```

Local Development

```
uvicorn app.main:app --reload
```

Docker / Deployment

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

What does `0.0.0.0` mean?

It means:

| Listen on all network interfaces.

It allows traffic from outside the Docker container to reach Uvicorn.

| ⚠ You normally browse to `localhost:8000`, not `0.0.0.0:8000`.



6. JSON

JSON is the main format used to exchange data through APIs.

Example:

```
{
  "feedback": "Delivery was late."
}
```

Very similar to a Python dictionary:

```
{
    "feedback": "Delivery was late."
}
```

Why JSON?

Because applications written in different programming languages can all understand it.

7. Pydantic

What is Pydantic?

Pydantic validates and structures incoming and outgoing data.

Example:

```
class FeedbackRequest(BaseModel):  
    feedback: str
```

This means:

| The request must contain a `feedback` field and it must be a string.

Adding Validation

```
feedback: str = Field(  
    ...,  
    min_length=3,  
    max_length=2000  
)
```

Meaning

Rule	Meaning
<code>...</code>	Required
<code>str</code>	Must be text
<code>min_length=3</code>	At least 3 characters
<code>max_length=2000</code>	Maximum 2,000 characters

Bad request:

```
{  
    "feedback": ""
```

```
}
```

FastAPI rejects it before it reaches the ML code.

Typical response:

```
422 Unprocessable Entity
```

Remember

Validation protects the model and business logic from malformed input.



8. Request and Response Schemas

Request

```
class FeedbackRequest(BaseModel):  
    feedback: str
```

Response

```
class FeedbackAnalysisResponse(BaseModel):  
    sentiment: str  
    category: str  
    priority: str
```

Together, they form an **API contract**.

```
Client promises:  
"I will send feedback."  
  
API promises:  
"I will return sentiment, category and priority."
```


9. APIRouter

Instead of putting every endpoint in `main.py`, we separated related routes.

```
router = APIRouter(  
    prefix="/feedback",  
    tags=["Feedback"]  
)
```

Then:

```
@router.post("/analyse")
```

becomes:

```
/feedback + /analyse  
  
= /feedback/analyse
```

The router is added to FastAPI using:

```
app.include_router(feedback_router)
```

Why use routers?

As applications grow:

```
main.py  
├─ feedback  
├─ users  
├─ models  
├─ admin  
├─ metrics  
└─ health
```

would become messy.

Routers keep related APIs organised.



10. Separation of Concerns

One of the biggest software-engineering concepts we learned.

Instead of:

```
main.py
  ↓
everything
```

we separated responsibilities.

File	Responsibility
<code>main.py</code>	Create application
<code>schemas.py</code>	Request/response validation
<code>api/feedback.py</code>	HTTP endpoint
<code>feedback_service.py</code>	Business logic
<code>sentiment_model.py</code>	ML inference
<code>train_sentiment.py</code>	Model training

Why?

- easier to understand
- easier to debug
- easier to test
- easier to replace components
- easier to extend



Key principle

Each component should have one clear responsibility.



11. Machine Learning Problem

Our sentiment task was:

Supervised Binary Classification

Input:

```
customer review
```

Output:

```
positive  
negative
```

Examples:

```
"I love this product." → positive  
"This product is terrible." → negative
```

The model learns from reviews where the correct sentiment label is already known.



12. Dataset

We used the **Cleanlab Amazon Reviews** dataset.

Training Set

```
5,000 reviews
```

Original distribution:

Label	Count
-------	-------

Positive	3,092
Negative	1,908

Test Set

1,000 reviews

Columns:

review_text
label

There were no missing values.

13. Why Did We Inspect the Data First?

We checked:

```
train_df.shape  
train_df.columns  
train_df.head()  
train_df.isnull().sum()  
train_df["label"].value_counts()
```

Questions data inspection answers

- How many records do we have?
- Which columns exist?
- Are there missing values?
- Are the classes balanced?
- What does the text actually look like?
- Do labels look correct?

✗ Bad ML workflow:

`download → model.fit()`

✓ Better workflow:

`download → understand → prepare → train`



14. Class Imbalance

Original:

```
Positive = 3092
Negative = 1908
```

The positive class was larger.

We downsampled the positive class:

```
Positive = 1908
Negative = 1908
```

Why?

A classifier can sometimes favour the majority class.

Balancing gives both classes equal representation during training.

Why Didn't We Balance the Test Set?

Because the test set should represent **unseen real data**.

```
Training data
→ we can modify / balance
```

```
Test data
→ keep untouched
```



Exam point

Never manipulate the test set just to improve model metrics.



15. TF-IDF

Full Form

Term Frequency — Inverse Document Frequency

ML algorithms need numbers, not raw text.

Input:

```
"The product is terrible"
```

TF-IDF converts that into a numerical vector.

Conceptually:

```
terrible → 0.72  
product  → 0.31  
is       → 0.03
```

Term Frequency

Measures how frequently a term occurs in a document.

Inverse Document Frequency

Reduces the importance of terms appearing in almost every document.

For example:

```
the  
is  
and
```

are generally less informative than:

```
terrible  
refund  
excellent  
broken
```



16. N-Grams

We used:

```
ngram_range=(1, 2)
```

This means both:

Unigrams

```
bad  
good  
refund  
broken
```

Bigrams

```
very good  
not working  
charged twice  
great product
```

Why bigrams?

Compare:

```
working
```

with:

```
not working
```

The second contains much more useful context.



17. `min_df=2`

We used:

```
min_df=2
```

Meaning:

| Ignore a term if it occurs in fewer than two documents.

This can remove:

- rare typos
 - one-off names
 - irrelevant unique words
-



18. Logistic Regression

Although its name says **Regression**, Logistic Regression is widely used for classification.

Our flow:

```
TF-IDF features
  ↓
Logistic Regression
  ↓
positive / negative
```


Why did we choose it?

- lightweight
- fast
- strong baseline for NLP
- works well with sparse TF-IDF features
- easy to deploy
- small computational cost

💡 We did not need BERT just because BERT exists.

Start with a good baseline and add complexity only if evaluation justifies it.

19. Scikit-Learn Pipeline

We combined preprocessing and classification:

```
Pipeline([
    ("tfidf", TfidfVectorizer(...)),
    ("classifier", LogisticRegression(...))
])
```

Flow:

```
Raw Text
  ↓
TF-IDF
  ↓
Logistic Regression
  ↓
Prediction
```

Why is Pipeline useful?

Without it:

```
vectors = tfidf.transform(text)
classifier.predict(vectors)
```

With it:

```
model.predict(text)
```

It keeps preprocessing and classification together.



20. Training

The main training command:

```
model.fit(X_train, y_train)
```

Before `fit()`

The model has not learned anything.

During `fit()`

The algorithm learns relationships between TF-IDF features and labels.

After `fit()`

It can predict unseen examples.



21. Model Evaluation

Our model achieved approximately:

Metric	Result
Accuracy	95.1%
Macro F1	95.0%
Negative F1	94.4%

Positive F1	95.7%
-------------	-------

Negative-class metrics:

```
Precision = 96.71%
Recall    = 92.17%
F1        = 94.39%
```

Positive-class metrics:

```
Precision = 93.90%
Recall    = 97.47%
F1        = 95.65%
```

22. Accuracy

Question:

| Out of all predictions, how many were correct?

Formula:

```
Correct Predictions
-----
Total Predictions
```

95.1% on 1,000 test samples means roughly:

```
951 predictions correct
49 incorrect
```

23. Precision

Question:

| When my model predicts this class, how often is it correct?

Formula:

$$\frac{TP}{TP + FP}$$

High precision → fewer false positives.

24. Recall

Question:

| Out of all real examples of this class, how many did I detect?

Formula:

$$\frac{TP}{TP + FN}$$

High recall → fewer missed cases.

25. F1 Score

Balances precision and recall.

$$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Useful when both false positives and false negatives matter.

26. Macro F1

Calculate F1 for every class separately, then average them equally.

Why useful?

Because one large class cannot dominate the score.



27. Confusion Matrix

Shows exactly where predictions went wrong.

	Predicted	
	Negative	Positive
Actual Negative	✓	✗
Actual Positive	✗	✓

It gives more information than accuracy alone.



28. Saving the Model — Joblib

We saved our trained pipeline:

```
joblib.dump(  
    model,  
    "models/sentiment_model.joblib"  
)
```

Later the API loads it:

```
joblib.load(  
    "models/sentiment_model.joblib"  
)
```

Why?

Without saving:

```
API starts
↓
train model again ❌
```

With Joblib:

```
Train once
↓
Save artifact
↓
API loads it
↓
Predict
```

29. Inference

Inference means:

| Using a trained model to make predictions on new data.

```
model.predict(
    ["The service was terrible."]
)
```

Output:

```
negative
```

? 30. Why `model.predict([text])` ?

Scikit-learn expects a collection of samples.

One sample:

```
[  
  "Terrible service"  
]
```

Three samples:

```
[  
  "Terrible service",  
  "Amazing product",  
  "I love it"  
]
```



31. Category Classification

Currently **rule-based**, not ML.

Examples:

```
refund / payment / charged  
    ↓  
  billing
```

Categories:

- billing
- technical
- customer_support
- delivery
- general



Future improvement:

Train a multi-class ML category classifier.

32. Priority Detection

Also currently rule-based.

Possible high-priority phrases:

- `urgent`
- `charged twice`
- `cannot login`
- `account locked`
- `crash`

Why can rules make sense here?

Priority may be a **business policy**, not purely an NLP prediction problem.

Example:

```
"payment system down"
```

The company might decide this is always high priority.

33. Logging

What is logging?

Logging records what the application is doing.

Instead of:

```
print("Started")
```

we use:

```
logger.info("Starting feedback analysis")
```

Logging Levels

Level	Typical Meaning
DEBUG	Developer details
INFO	Normal events
WARNING	Something unusual
ERROR	Operation failed
CRITICAL	Serious application failure

Why Logging Matters

Once an app is deployed, logs help answer:

- Did the app start?
- Was the endpoint called?
- Did prediction succeed?
- Did an exception occur?
- Where did it fail?

Privacy Lesson

Don't blindly log:

```
Full customer message  
email  
account details  
names
```

Instead:

```
sentiment=negative  
category=billing  
priority=high
```

34. pytest

What is pytest?

A Python automated-testing framework.

Example:

```
def test_positive_feedback():
    result = analyse_feedback(
        "I love this product"
    )

    assert result["sentiment"] == "positive"
```

35. assert

```
assert result["sentiment"] == "negative"
```

means:

| I expect this condition to be true.

If true:

PASS 

If false:

FAIL 

36. Types of Tests We Used

Service Test

Directly tests:

```
analyse_feedback(...)
```

Purpose:

| Does the analysis logic work?

API Test

Uses:

```
TestClient(app)
```

Then:

```
response = client.post(...)
```

Purpose:

| Does the API behave correctly?

Validation Test

Send:

```
{  
  "feedback": ""  
}
```

Expect:

```
422
```

Purpose:

| Does invalid input get rejected?

37. Why Multiple Test Layers?

```
Service test
  → Is business logic correct?

API test
  → Does HTTP behaviour work?

Validation test
  → Are bad requests rejected?
```

If something breaks, this separation helps locate the problem.

38. Why `python -m pytest` ?

We first ran:

```
pytest -v
```

and it accidentally used Anaconda Python.

The traceback showed:

```
/opt/anaconda3/bin/python
```

Our packages were installed inside another virtual environment.

So we used:

```
python -m pytest -v
```

This tells the current Python interpreter:

| Run the pytest module.

This reduced environment confusion.



39. Virtual Environment

Create:

```
python3 -m venv venv
```

Activate:

```
source venv/bin/activate
```

Purpose:

```
Project A dependencies
≠
Project B dependencies
```

Each project can have its own package versions.



40. requirements.txt

Generated using:

```
pip freeze > requirements.txt
```

Another machine can then run:

```
pip install -r requirements.txt
```

Used by:

- developers
- Docker
- CI
- Render deployment

41. Docker

What is Docker?

Docker packages:

```
application
+ Python
+ dependencies
+ model
+ startup instructions
```

into a reproducible environment.

42. Image vs Container

```
Dockerfile
↓
docker build
↓
IMAGE
↓
docker run
```

↓
CONTAINER

Easy analogy

 **Image** = packaged template

 **Container** = running copy of that template

43. Dockerfile

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD [
    "uvicorn",
    "app.main:app",
    "--host",
    "0.0.0.0",
    "--port",
    "8000"
]
```

44. Dockerfile Commands

FROM

```
FROM python:3.10-slim
```

Start from a lightweight environment with Python already installed.

WORKDIR

```
WORKDIR /app
```

Similar to:

```
cd /app
```

inside the container.

COPY

```
COPY requirements.txt .
```

copies the file into the image.

```
COPY . .
```

copies the project.

RUN

```
RUN pip install ...
```

Runs **during image creation**.

CMD


```
CMD ["uvicorn", ...]
```

Runs **when the container starts**.

🎯 Very common exam/interview question:

RUN = build time

CMD = container run time

45. Building the Docker Image

```
docker build -t customer-feedback-api .
```

Part	Meaning
<code>docker build</code>	Build an image
<code>-t</code>	Give image a name/tag
<code>customer-feedback-api</code>	Image name
<code>.</code>	Current directory is build context

46. Running the Container

```
docker run -p 8000:8000 customer-feedback-api
```

Port mapping:

```
Laptop
8000
↓
Docker container
8000
```

Browser:

```
localhost:8000
```

47. **.dockerignore**

Examples:

```
venv/  
__pycache__/  
.pytest_cache/  
.git/  
.DS_Store  
*.pyc
```

Why?

Don't copy unnecessary local files into the Docker image.

Benefits:

- faster build
- smaller context
- cleaner image

48. Git

Git = version control.

Important commands:

```
git init  
git add .  
git commit  
git push
```

49. `git init`

```
git init
```

Creates:

```
.git/
```

That hidden directory stores Git history and metadata.

50. Git Staging

```
git add .
```

Flow:

```
Working files
  ↓
git add
  ↓
Staging area
  ↓
git commit
  ↓
Repository history
```

51. Commit

```
git commit -m "Add Docker configuration"
```

A commit is a saved project checkpoint.

Contains:

- changes
 - author
 - timestamp
 - commit message
-

52. Git Identity vs GitHub Login

This was an actual issue we encountered.

```
git config user.name  
git config user.email
```

controls:

| Who appears as the **commit author**.

It does NOT control:

| Which GitHub account authenticates the push.

These are separate concepts.

53. GitHub Authentication

GitHub no longer accepts normal account passwords for Git operations over HTTPS.

We used a:

Fine-grained Personal Access Token

The token was scoped to this repository instead of all repositories.

|  Principle: give credentials only the minimum permissions needed.

54. GitHub Actions

GitHub Actions automates workflows.

Workflow file:

```
.github/workflows/ci.yml
```

We used it for CI.

55. Continuous Integration — CI

CI = **Continuous Integration**

```
Push code
  ↓
GitHub Actions
  ↓
Fresh Ubuntu machine
  ↓
Install Python
  ↓
Install dependencies
  ↓
Run pytest
  ↓
PASS / FAIL
```

56. Why CI Is Important

Without CI:

| “It works on my laptop.”

With CI:

| “It also works in a fresh external environment.”

CI can expose hidden problems such as:

- missing dependencies
 - operating-system assumptions
 - failing tests
 - configuration issues
-

57. CD

CD can mean:

- Continuous Delivery
- Continuous Deployment

Our simplified workflow:

```
git push
  ↓
GitHub
  ↓
GitHub Actions CI
  ↓
Render sees new commit
  ↓
Docker rebuild
  ↓
New version deployed
```

58. Render Deployment

Render hosted our FastAPI Docker container.

```
GitHub Repository
  ↓
Render
  ↓
Dockerfile
  ↓
Build Docker Image
  ↓
```

```
Start Container
↓
Public API URL
```

This changed our API from:

```
localhost:8000
```

to a public service.



59. Local vs Deployment

Local	Deployment
Runs on your laptop	Runs on cloud server
127.0.0.1	Public URL
Mainly developer access	Internet accessible
Stops when your machine stops	Hosted remotely



60. Swagger / OpenAPI

FastAPI automatically generated:

```
/docs
```

Swagger shows:

- API endpoints
- request schemas
- response schemas
- HTTP methods
- status codes

It also lets us execute API requests through the browser.

61. Health Endpoint

```
GET /health
```

Response:

```
{
  "status": "healthy"
}
```

Purpose:

| Check whether the application is alive.

Used by:

- deployment systems
- monitoring
- load balancers
- orchestration systems

62. Why Did Initially Return 404?

Render requested:

```
GET /
```

We had not defined:

```
@app.get("/")
```


So FastAPI correctly returned:

```
404 Not Found
```

This did **not** mean deployment failed.

Our actual endpoints still worked:

```
/health  
/docs  
/feedback/analyse
```

63. Complete Request Flow

Suppose:

```
{  
  "feedback": "The product stopped working after two days."  
}
```

Flow:

```
1. User sends POST request  
    ↓  
2. FastAPI receives it  
    ↓  
3. Pydantic validates it  
    ↓  
4. Router calls service  
    ↓  
5. Service calls sentiment model
```

↓

6. TF-IDF converts text to numbers

↓

7. Logistic Regression predicts sentiment

↓

8. Category rules run

↓

9. Priority rules run

↓

10. Results combined

↓

11. Response schema validates output

↓

12. FastAPI returns JSON

64. Biggest Lesson From the Project

A machine-learning application is NOT just:

```
model.fit()  
model.predict()
```

A usable ML system also needs:

- APIs
- validation
- architecture
- testing

- logging
 - dependency management
 - Docker
 - version control
 - CI
 - deployment
 - documentation
-



65. Model Development vs Model Serving

Model Development

Questions:

- Which dataset?
- Which algorithm?
- Which features?
- How accurate?
- Which metric?

Model Serving

Questions:

- How does another system call it?
- How do we validate inputs?
- How do we load the model?
- How do we test the API?
- How do we package it?

- How do we deploy it?

This project taught us **both sides**.

66. Current Limitations

- sentiment only positive/negative
 - category is rule-based
 - priority is rule-based
 - model trained on Amazon reviews
 - domain shift may affect performance
 - no authentication
 - no persistent database
 - no monitoring yet
-

67. Future Improvements

Possible next steps:

- neutral sentiment
- ML category classifier
- confidence score
- human-review threshold
- batch prediction
- monitoring
- latency metrics
- model versioning
- drift detection
- LLM summaries

- database
 - authentication
 - rate limiting
-



68. Exam / Interview Quick Questions

What is FastAPI?

A Python framework for building APIs.

What is Uvicorn?

An ASGI server that runs the FastAPI application and listens for HTTP requests.

What is Pydantic?

A Python library for data validation and schema definition.

GET vs POST?

`GET` retrieves information.

`POST` sends data to the server for processing.

What is TF-IDF?

A numerical representation of text based on term frequency and how rare a term is across documents.

What is Logistic Regression?

A supervised learning algorithm commonly used for classification.

What does `fit()` do?

Learns model parameters from training data.

What is inference?

Using a trained model to predict unseen data.

Why save the model?

So the deployed application can load the model instead of retraining it.

What is Joblib?

A Python library commonly used to persist fitted scikit-learn objects.

What is precision?

Of everything predicted as a class, how many were correct?

What is recall?

Of all actual examples of a class, how many were found?

What is F1?

The harmonic mean of precision and recall.

What is pytest?

A Python framework for automated testing.

What is `assert` ?

It checks whether an expected condition is true.

What is Docker?

A platform for packaging applications and dependencies into reproducible environments.

Image vs container?

Image = template.

Container = running instance.

`RUN` vs `CMD` in Docker?

`RUN` executes while building the image.

`CMD` executes when the container starts.

Why 0.0.0.0?

Allows the server inside Docker to listen on all interfaces.

What is CI?

Automatically validating code changes, typically through tests.

What is GitHub Actions?

GitHub's workflow automation platform.

What is deployment?

Running the application on remote infrastructure so others can access it.

69. Commands Cheat Sheet

Virtual Environment

```
python3 -m venv venv  
source venv/bin/activate
```

Dependencies

```
pip install -r requirements.txt
```

FastAPI

```
uvicorn app.main:app --reload
```

Tests

```
python -m pytest -v
```

Docker Build

```
docker build -t customer-feedback-api .
```

Docker Run

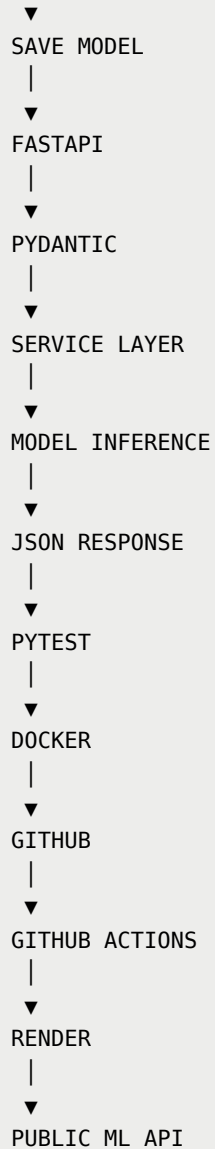
```
docker run -p 8000:8000 customer-feedback-api
```

Git

```
git status  
git add .  
git commit -m "message"  
git push
```

70. One Diagram to Remember Everything

```
DATA  
|  
▼  
INSPECT  
|  
▼  
PREPARE  
|  
▼  
TRAIN  
|  
▼  
EVALUATE  
|
```

Final Takeaway

A trained model is not the same thing as an AI application.

The model performs the prediction.

FastAPI makes it accessible.

Pydantic protects the interface.

pytest verifies that it works.

Logging helps observe it.
Docker makes it reproducible.
Git tracks its evolution.
GitHub Actions automatically validates changes.
Render makes it accessible to users.

The complete learning journey

```
Machine Learning
+
Backend Engineering
+
Testing
+
Containerisation
+
Version Control
+
Automation
+
Cloud Deployment
=
End-to-End ML Engineering
```